Report

Results:Queries

Query Q1: Count distinct values for each column.

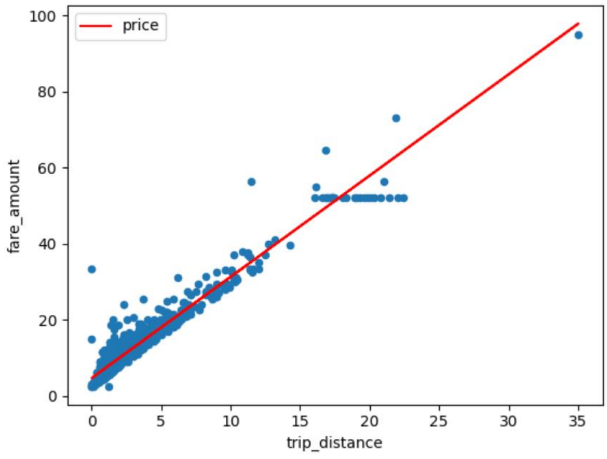
Times:
Pandas: 7.593414306640625
SQLite: 59.94462823867798
DuckDB: 6.420370817184448

Query Q2: Compare the performance of the aggregation/grou ping functions.

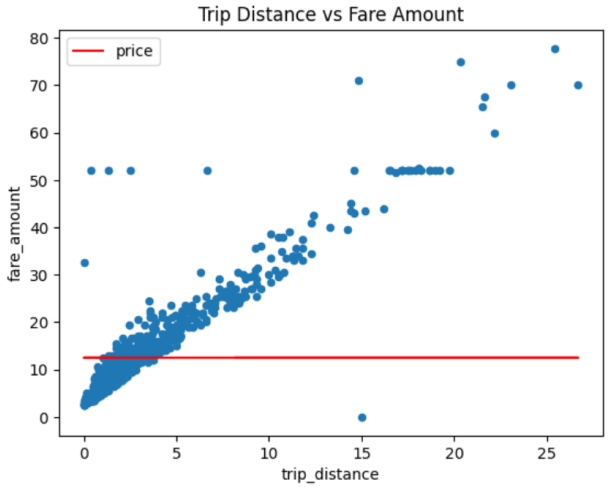
Times:
Pandas: 1.0965170860290527
SQLite: 20.572500228881836
DuckDB: 0.438723087310791

Results: Machine Learning - Fare Estimation

SQLite:
Alpha: 4.65161699157053
Beta: 2.661441728279978
ML Regression in SQLite: 16.061692476272583



DuckDB:
Alpha: 12.487179884846261
Beta: 4.985959270135805e-06
ML Regression in DuckDB: 0.6965370178222656



Experiments:

1. Count distinct values for each column: Simply using SQL query selects multiple fields from the yellow_tripdata_2016_01 dataset and counts the number of distinct values for each field (COUNT(DISTINCT ...)).

2. Compare the performance of the aggregation/grouping functions:

In SQlite way, a temporary table event_counts1 is created through the WITH clause to count the number of taxi orders per day and per hour. The strftime() function is used to extract the day of the year and hour of the day from the tpep_pickup_datetime field. Finally, the MIN, AVG and MAX values of event_count are counted.

In DuckDB way, We use the EXTRACT() function to extract the day and hour.

3. Machine Learning : Fare Estimation

SQLite: By using SQL query, 1000 records of trip_distance and fare_amount are randomly selected for drawing scatter plots. This is achieved through random sampling with ORDER BY RANDOM(). Then, use pd.read_sql_query() method to convert the query results into a DataFrame for subsequent processing and plotting.

DuckDB: Inspired by SQLites' method, We use STDDEV_SAMP() & COVAR_SAMP() to calculate the mean and variance of the data. Then, 1000 data are randomly selected for subsequent scatter plot drawing similarly. Using the query() method of the DuckDB Python client to execute SQL queries and convert the results to a Pandas DataFrame for subsequent data processing and plotting.

Observations:

1.Potential reasons for the time differences between three systems in querying:

DuckDB's columnar storage, parallel execution, and modern query optimization techniques make it the best performer for large-scale data analysis; At the same time, Pandas relies on memory operations and performs well on small and medium-sized data, but is not as optimized as DuckDB for analytical queries. However, SQLite's disk I/O operations, lack of parallelization, and optimization lead to poor performance when the data scale is large.

2.Potential reasons for the time differences between SQLite and DuckDB in linear regression:

DuckDB's architecture like column storage, query optimization, parallelization support make it significantly faster than SQLite when processing analytical queries. The lack of efficient query optimization makes SQLite significantly slower when performing complex analysis tasks.

DuckDB is able to store data and calculation results in memory when processing such tasks, while using parallel computing to improve efficiency, thereby significantly shortening the running time.